

前言

笔者主要想分享一些基础的字符串，由于笔者也是现学的，所以难免会犯各种错误，如有不当之处，敬请指正。

同时，笔者在写讲义的过程中发现自己**完全**不擅长教学，所以本讲义含有大量的复制粘贴，面向AI，以及直接贴链接，敬请见谅。

字符串哈希

引入

我们现在有2个同样长的字符串，我们希望比较它们是否相同，该怎么办呢？

最直观的方法，就是一个字符一个字符地比较，时间复杂度 $O(len)$ ， len 为字符串的长度。

那如果现在有 n 个字符串，希望找到所有相同的字符串呢？

如果依旧按照朴素方法，每次比较都要一个一个字符地比，一旦字符串比较长，复杂度就会很大。

那么，有没有 $O(1)$ 比较字符串的方法呢？

有的，兄弟有的。

哈希函数

我们知道，比较两个整数是否相同的时间复杂度为 $O(1)$ ，那么如果可以把不同的字符串对应不同的整数，我们就只需要比较对应的数就行了。

而哈希函数，就是一种把字符串映射到整数的函数。

我们不妨先考虑只有单个字符的情况，可以想到，直接使用字符对应的 $ASCII$ 码就行了。

那么现在面对字符串了，我们希望有一种办法，可以将几个数组合成一个数，且互不影响。（或许）可以想到，直接把数字拼在一起就行了。

于是我们得到了最常用的多项式 Hash 函数：

$$f(s) = \sum_{i=1}^l s[i] \cdot b^{l-i} \pmod{M}$$

(b 大于所有 $s[i]$ 的 $ASCII$ 码最大值， M 为大质数)

我们可以把它理解为一个长为 l 的 b 进制数。这个 b 进制数的第 i 位，就是字符串中的第 i 位的 $ASCII$ 码。

如果先不考虑取模，那么

两个字符串相同 $\Leftrightarrow$

两个字符串中的字符都相同 $\Leftrightarrow$

两个 b 进制数的各位都相同 $\Leftrightarrow$

两个 b 进制数相同

同理，两个字符串不相同 $\Leftrightarrow$ 两个 b 进制数不相同

哈希碰撞（冲突）

但是，由于得到的数可能很大，所以我们不得不为这个数取模，这就使得两个不同的字符串的 $Hash$ 值可能相同。

我们将 $Hash$ 值相同但原字符串不相同的现象称为**哈希碰撞**。

值得区分的是，虽然取模导致了 $Hash$ 值相同的时候，两个字符串不一定相同（但是大概率相同）。

但是 $Hash$ 值不相同，取模之前的数一定也不相同，继而两个字符串肯定不相同。

为避免哈希碰撞，我们希望模数 M 有以下特征：

1. 大，这样区间更大，能生成的不同哈希值更多；
2. $M \in \mathbb{P}$ ，这样 M 的因子最少，取模后结果相等的可能性最小。

所以，我们一般选取998244353， $1e9 + 7$ 这样的数作为模数。

还有一种写法，为了方便和扩大模数，我们可以使用unsigned long long来定义 $Hash$ 函数的结果，不进行取模。这种写法被称为**自然溢出**。

ull在 $2^{64} - 1$ 再+1后会溢出，直接从0重新开始，所以我们相当于把模数 M 定为 2^{64} ，这也是一个不错的选择。

参考资料

$Hash$ 的技巧还有很多，感兴趣的可以看看以下参考资料。

[OI Wiki 字符串哈希](#)

第一题：P3370【模板】字符串哈希

模板题喵。

```
1  const ll MOD = 998244353;
2  const ll base = 191;
3  ll has(string &s){
4      ll len=s.size();
5      ll ans=0;
6      for(char c:s){
7          ans=(ans*base+c-'a'+1)%MOD;
8      }
9      return ans;
10 }
11
12 void solve(){
13     ll n;
14     cin>>n;
15     vector<ll> vec;
16     for(int i=1;i<=n;++i){
17         string s;
18         cin>>s;
19         vec.push_back(has(s));
20     }
21     sort(vec.begin(),vec.end());
22     vec.erase(unique(vec.begin(),vec.end()),vec.end());
23     cout<<vec.size();
```

第二题：P3879 阅读理解

set太好用了你们知道吗？

```
1  const ll MOD = 998244353;
2  const ll base = 191;
3  ll has(string s){
4      ll len=s.size();
5      ll ans=0;
6      for(char c:s){
7          ans=(ans*base+c-'a'+1)%MOD;
8      }
9      return ans;
10 }
11
12 void solve(){
13     ll n;
14     cin>>n;
15     vector<set<ll>> a(n+1);
16     for(int i=1;i<=n;++i){
17         ll l;
18         cin>>l;
19         for(int j=1;j<=l;++j){
20             string s;
21             cin>>s;
22             a[i].insert(has(s));
23         }
24     }
25     ll m;
26     cin>>m;
27     while(m--){
28         string s;
29         cin>>s;
30         ll t=has(s);
31         for(int i=1;i<=n;++i){
32             if(a[i].find(t)!=a[i].end()){
33                 cout<<i<<" ";
34             }
35         }
36         cout<<"\n";
37     }
38 }
```

字典树

参考资料

[Trie树的可视化操作](#)

[字典树 \(Trie\) - OI Wiki](#)

第三题: P8306【模板】字典树

我这个纯为了做题写的, 板子的话还是建议学习丁冠, 搞结构体封装。

```
1  vector<map<ll,ll>> nxt(N);
2  vector<ll> vis(N);
3
4  void solve(){
5      ll n,q;
6      cin>>n>>q;
7      ll cnt=1;
8
9      for(int i=1;i<=n;++i){ //建树
10         string s;
11         cin>>s;
12         ll len=s.size();
13         s=" "+s;
14         ll tmp=0;
15         for(int j=1;j<=len;++j){
16             ll x=s[j]-'0';
17             if(!nxt[tmp][x]){
18                 nxt[tmp][x]=cnt;
19                 cnt++;
20             }
21             tmp=nxt[tmp][x];
22             vis[tmp]++;
23         }
24     }
25
26     while(q--){ //查询
27         string s;
28         cin>>s;
29         ll len=s.size();
30         s=" "+s;
31         ll tmp=0;
32         for(int i=1;i<=len;++i){
33             ll x=s[i]-'0';
34             if(!nxt[tmp][x]){
35                 cout<<0<<"\n";
36                 break;
37             }
38             tmp=nxt[tmp][x];
39             if(i==len){
40                 cout<<vis[tmp]<<"\n";
41                 break;
42             }
43         }
44     }
45
46     for(int i=0;i<=cnt-1;++i){
47         nxt[i].clear();
```

```

48     vis[i]=0;
49 }
50 }

```

第四题： [P10471 最大异或对](#)

对于一个确定的数，要找和它异或结果最大的数，我们只需要从它的二进制最高位开始，如果能取相反的位，那么肯定是取了的结果比不取的更大，如果不能取才取相同的位。

而取尽可能取相反的位这个操作正好可以用字典树来实现。

考虑将 $A_1, A_2, \dots, A_n$ 转化为31位的2进制数，按照从位数从大到小插入字典树。

然后对于每个 A_i ，我们都希望在每一位上尽可能取与 A_i 该位相反的数。对应字典树上，就是判断由当前位置指向与 A_i 该位相反的数的路径是否存在，存在就选相反的数，不存在就取相同的数。通过不断地重复这样的操作，我们可以找出与 A_i 异或结果最大的数，继而得到选了 A_i 的最大的异或结果。

n 个结果取最大值即答案。

```

1  vector<vector<ll>> nxt(N,vector<ll>(2,0));
2  void solve(){
3      ll n;
4      cin>>n;
5      vector<vector<int>> a(n+1,vector<int>(32));
6      for(int i=1;i<=n;++i) {
7          ll x;
8          cin>>x;
9          for(int j=31;j>=1;--j){
10             a[i][j]=x&1;
11             x>>=1;
12         }
13     }
14     ll cnt=1;
15     for(int i=1;i<=n;++i){ //建树
16         ll tmp=0;
17         for(int j=1;j<=31;++j){
18             ll x=a[i][j];
19             if(!nxt[tmp][x]){
20                 nxt[tmp][x]=cnt;
21                 cnt++;
22             }
23             tmp=nxt[tmp][x];
24         }
25     }
26 }
27
28 ll ans=0;
29 for(int i=1;i<=n;++i){
30     ll res=0;
31     ll tmp=0;
32     for(int j=1;j<=31;++j){
33         ll x=a[i][j];
34         if(nxt[tmp][1-x]){

```

```

35         res+=(1LL<<(31-j));
36         tmp=nxt[tmp][1-x];
37     }
38     else{
39         tmp=nxt[tmp][x];
40     }
41 }
42 ans=max(ans,res);
43 }
44 cout<<ans;
45 }

```

第五题：P4551 最长异或路径

由于异或的神秘小性质 $a \oplus a = 0$ ，（或许）可以想到， x 和 y 之间的路径的异或值其实等于 1和 x 之间的路径的异或值 异或上 1和 y 之间的路径的异或值。因为这样重复的路径会抵消掉。

然后就变成和第四题一样了，先跑一遍dfs算出1到每个点的路径异或值，然后就是找这些里面的最大异或对。

```

1  vector<vector<pll>> G(N);
2  vector<ll> dis(N);
3  vector<bool> vis(N);
4
5  void dfs(ll u,ll x){
6      vis[u]=1;
7      dis[u]=x;
8      for(auto [v,w]:G[u]){
9          if(vis[v]) continue;
10         dfs(v,(x^w));
11     }
12 }
13
14 void solve(){
15     ll n;
16     cin>>n;
17     for(int i=1;i<=n-1;++i){
18         ll u,v,w;
19         cin>>u>>v>>w;
20         G[u].push_back({v,w});
21         G[v].push_back({u,w});
22     }
23     dfs(1,0);
24     vector<vector<int>> a(n+1,vector<int>(32));
25     for(int i=1;i<=n;++i) {
26         ll x=dis[i];
27         for(int j=31;j>=1;--j){
28             a[i][j]=(x&1);
29             x>>=1;
30         }
31     }
32 }
33
34 vector<vector<ll>> nxt(n*31,vector<ll>(2,0));

```

```

35     ll cnt=1;
36     for(int i=1;i<=n;++i){ //建树
37         ll tmp=0;
38         for(int j=1;j<=31;++j){
39             ll x=a[i][j];
40             if(!nxt[tmp][x]){
41                 nxt[tmp][x]=cnt;
42                 cnt++;
43             }
44             tmp=nxt[tmp][x];
45         }
46     }
47
48     ll ans=0;
49     for(int i=1;i<=n;++i){
50         ll res=0;
51         ll tmp=0;
52         for(int j=1;j<=31;++j){
53             ll x=a[i][j];
54             if(nxt[tmp][1-x]){
55                 res+=(1LL<<(31-j));
56                 tmp=nxt[tmp][1-x];
57             }
58             else{
59                 tmp=nxt[tmp][x];
60             }
61         }
62         ans=max(ans,res);
63     }
64     cout<<ans;
65 }

```

KMP

参考资料

本来 KMP 想自己写的，但我写着写着发现 KMP 不配图的话我确实讲的不怎么清楚，建议大家都看参考资料喵。

[非常好的视频](#)

[以及一个很好的俄语博文的翻译](#)

第六题:【模板】KMP

```

1  vector<ll> ans;
2
3  void kmp(string a,string b){
4      string s=" "+b+"#"+a;
5      vector<ll> pi(s.size(),0);
6      for(int i=2;i<=s.size()-1;++i){
7          ll len=pi[i-1];
8          while(len and s[i]!=s[len+1]){

```

```

9         len=pi[len];
10    }
11    if(s[i]==s[len+1]){
12        pi[i]=len+1;
13    }
14    if(pi[i]==b.size()){
15        ans.push_back(i-2*b.size());
16    }
17 }
18 }
19
20 void solve(){
21     string a,b;
22     cin>>a>>b;
23     kmp(a,b);
24     for(auto &p:ans) cout<<p<<"\n";
25     b=" "+b;
26     vector<ll> pi(b.size(),0);
27     cout<<0<<" ";
28     for(int i=2;i<=b.size()-1;++i){
29         ll len=pi[i-1];
30         while(len and b[i]!=b[len+1]){
31             len=pi[len];
32         }
33         if(b[i]==b[len+1]){
34             pi[i]=len+1;
35         }
36         cout<<pi[i]<<" ";
37     }
38 }

```

第七题:[P9606 ABB](#)

容易想到，只要求出了包含最后一个字符的最长回文串的长度 len ， $n - len$ 即是答案。

那么怎么求呢？

考虑将字符串 s 反转得到 t ，拼接字符串 $t+\#+s$ 得到 k ， k 的最后一个字符的前缀函数的值，就是包含最后一个字符的最长回文串的长度。

有点绕，举个栗子。比如 s 的长度为4，我们设 s 为abcd,那么生成的 k 为dcba#abcd，为了方便，我们称 k 的最后一个字符为 x 。然后比如我们得到 x 处的前缀函数的值为3，说明dcb和bcd是相同的，即bcd是回文串,这说明了存在性。然后如果还有更长的回文，那么 x 处的前缀函数的值会大于3，矛盾，所以包含 x 的最长回文串的长度就是 x 处的前缀函数的值。

但我个人认为这个KMP的做法不太好想，而用下一节的Manacher则会好想很多，因为Manacher本身就是求回文串的算法。

```

1 vector<ll> ans;
2 ll maxn;
3
4 void kmp(string a,string b){
5     string s=" "+b+"#"+a;

```



```

6     vector<ll> pi(s.size(),0);
7     for(int i=2;i<=s.size()-1;++i){
8         ll len=pi[i-1];
9         while(len and s[i]!=s[len+1]){
10             len=pi[len];
11         }
12         if(s[i]==s[len+1]){
13             pi[i]=len+1;
14         }
15         if(pi[i]==b.size()){
16             ans.push_back(i-2*b.size());
17         }
18     }
19     maxn=pi[s.size()-1];
20 }
21
22 void solve(){
23     ll n;
24     cin>>n;
25     string s,t;
26     cin>>s;
27     t=s;
28     reverse(t.begin(),t.end());
29     kmp(s,t);
30     cout<<n-maxn;
31 }

```

第八题: [P4824 \[USACO15FEB\] Censoring S](#)

稍微修改一下 *KMP* 的板子, 因为会有删除, 所以改用类似栈的形式写 (不用栈是因为栈只能取栈顶的值)。每次 `pi[tmp]==b.size()` 就回退。

```

1 void kmp(string a,string b){
2     string s1=" "+b+"#" +a;
3     vector<ll> pi(2,0);
4     vector<char> s(1);
5     s.push_back(s1[1]);
6     ll tmp=1;
7     for(int i=2;i<=s1.size()-1;++i){
8         ll len=pi[tmp];
9         tmp++;
10        while(len and s1[i]!=s1[len+1]){
11            len=pi[len];
12        }
13        if(s1[i]==s1[len+1]){
14            pi.push_back(len+1);
15        }
16        else{
17            pi.push_back(0);
18        }
19        s.push_back(s1[i]);
20        if(pi[tmp]==b.size()){

```

```

21         for(int i=1;i<=b.size();++i){
22             pi.pop_back();
23             s.pop_back();
24         }
25         tmp-=b.size();
26     }
27 }
28 for(int i=b.size()+2;i<=s.size()-1;++i) cout<<s[i];
29 }
30
31 void solve(){
32     string a,b;
33     cin>>a>>b;
34     kmp(a,b);
35 }

```

Manacher

参考资料

[OI Wiki Manacher](#)

第九题:P3805 【模板】Manacher

```

1  void solve(){
2      string s1;
3      cin>>s1;
4      int len=s1.size();
5      vector<int> p(2*len+2);
6      vector<char> s(2*len+2);
7      s[0]=' ';
8      s[1]='*';
9      for(int i=0;i<=len-1;++i){
10         s[2+2*i]=s1[i];
11         s[3+2*i]='*';
12     }
13     int maxpos=0;
14     int mid=0;
15     for(int i=1;i<=2*len+1;++i){
16         if(maxpos>=i){
17             ll k=2*mid-i;
18             p[i]=min(p[k],maxpos-i+1);
19         }
20         else{
21             p[i]=1;
22         }
23         while(i-p[i]>=1 and i+p[i]<=len*2+1 and s[i-p[i]]==s[i+p[i]]){
24             p[i]++;
25         }
26         if(i+p[i]-1>maxpos){
27             maxpos=i+p[i]-1;

```

```

28         mid=i;
29     }
30 }
31 int ans=0;
32 for(int i=1;i<=2*len+1;++i){
33     ans=max(ans,(2*p[i]-1)/2);
34 }
35 cout<<ans;
36 }

```

第七题:P9606 ABB

回头一看，第六题用*Manacher*确实直观好想嘛。

现在只要先跑一遍*Manacher*，然后对于每个回文中心，判断以它为中心的回文串是否能达到最后一个字符，第一个能达到的就是最长回文串的长度。

```

1 void solve(){
2     int len;
3     cin>>len;
4     string s1;
5     cin>>s1;
6     vector<int> p(2*len+2);
7     vector<char> s(2*len+2);
8     s[0]=' ';
9     s[1]='*';
10    for(int i=0;i<=len-1;++i){
11        s[2+2*i]=s1[i];
12        s[3+2*i]='*';
13    }
14    int maxpos=0;
15    int mid=0;
16    for(int i=1;i<=2*len+1;++i){
17        if(maxpos>=i){
18            ll k=2*mid-i;
19            p[i]=min(p[k],maxpos-i+1);
20        }
21        else{
22            p[i]=1;
23        }
24        while(i-p[i]>=1 and i+p[i]<=len*2+1 and s[i-p[i]]==s[i+p[i]]){
25            p[i]++;
26        }
27        if(i+p[i]-1>maxpos){
28            maxpos=i+p[i]-1;
29            mid=i;
30        }
31    }
32    int ans=len;
33    for(int i=len;i<=2*len+1;++i){
34        if(i+p[i]-1>=2*len){
35            ll x=(2*p[i]-1)/2;
36            ans=len-x;

```

```

37         break;
38     }
39 }
40 cout<<ans;
41 }

```

第十题： [P1659 \[国家集训队\] 拉拉队排练](#)

先跑一遍`Manacher`。

现在我们已经有了所有的回文串了，下一个目标就是找出最长的 k 个。

由于只要奇数长度的回文串，所以我们只用考虑回文串中心在原字符串上的回文串就行。

对于回文串中心 x ，如果最长回文串的半径长度为 r ，那么该中心总共可以贡献长度为 $2r-1, 2r-3, \dots, 1$ 的回文串。观察这种形式，可以想到用差分来解决。对于每个中心，`diff[2*r-1]++`。然后对于差分数组，从最长开始做每次 -2 的后缀和，每次得到的就是当前长度的个数。不断相乘，直到总个数达到 k 。

```

1  const ll MOD = 19930726;
2
3  ll qpow(ll a, ll b){
4      ll res=1;
5      while(b){
6          if(b%2) res=res*a%MOD;
7          b>>=1;
8          a=a*a%MOD;
9      }
10     return res;
11 }
12
13 void solve(){
14     ll n,k;
15     cin>>n>>k;
16     string s1;
17     cin>>s1;
18     int len=s1.size();
19     vector<int> p(2*len+2);
20     vector<char> s(2*len+2);
21     s[0]=' ';
22     s[1]='*';
23     for(int i=0;i<=len-1;++i){
24         s[2+2*i]=s1[i];
25         s[3+2*i]='*';
26     }
27     int maxpos=0;
28     int mid=0;
29     for(int i=1;i<=2*len+1;++i){
30         if(maxpos>=i){
31             ll k=2*mid-i;
32             p[i]=min(p[k], maxpos-i+1);
33         }
34         else{
35             p[i]=1;

```

```

36     }
37     while(i-p[i]>=1 and i+p[i]<=len*2+1 and s[i-p[i]]==s[i+p[i]]){
38         p[i]++;
39     }
40     if(i+p[i]-1>maxpos){
41         maxpos=i+p[i]-1;
42         mid=i;
43     }
44 }
45 ll cnt=0;
46 ll maxn=0;
47 vector<ll> diff(N);
48 for(int i=2;i<=2*len;i=i+2){
49     ll x=(2*p[i]-1)/2;
50     cnt+=x;
51     diff[x]++;
52     maxn=max(maxn,(ll)(2*p[i])-1);
53 }
54 if(cnt<k) cout<<-1<<"\n";
55 else{
56     ll ans=1;
57     ll tmp=0;
58     for(int i=maxn;i>=1;i=i-2){
59         tmp+=diff[i];
60         if(k>tmp){
61             ans=ans*qpow(i,tmp)%MOD;
62             k-=tmp;
63         }
64         else{
65             ans=ans*qpow(i,k)%MOD;
66             break;
67         }
68     }
69     cout<<ans;
70 }
71 }

```

第十一题: [P4555 \[国家集训队\] 最长双回文串](#)

依旧先跑一遍 *Manacher*。

考虑对于每个点，找它左边的最长回文串和右边的最长回文串。最长回文串又可以转化为回文串中心离该点最远。那么只需要对于每个回文串中心，在它最远达到的左右点处更新最远中心（当然如果不如原来远就不变了）。记得这样做之后要对右边最远跑一次前缀max,即 $right[i]=\max(right[i-1],right[i])$ ，因为我们只对最远的点更新了，而实际上右边的一个中心都能到你的左边了，当然更能到你这个点。左边同理也要跑一次后缀min。

```

1 void solve(){
2     string s1;
3     cin>>s1;
4     int len=s1.size();
5     vector<int> p(2*len+2);
6     vector<char> s(2*len+2);

```

```

7   s[0]=' ';
8   s[1]='*';
9   for(int i=0;i<=len-1;++i){
10      s[2+2*i]=s1[i];
11      s[3+2*i]='*';
12  }
13  int maxpos=0;
14  int mid=0;
15  for(int i=1;i<=2*len+1;++i){
16      if(maxpos>=i){
17          ll k=2*mid-i;
18          p[i]=min(p[k],maxpos-i+1);
19      }
20      else{
21          p[i]=1;
22      }
23      while(i-p[i]>=1 and i+p[i]<=len*2+1 and s[i-p[i]]==s[i+p[i]]){
24          p[i]++;
25      }
26      if(i+p[i]-1>maxpos){
27          maxpos=i+p[i]-1;
28          mid=i;
29      }
30  }
31  vector<ll> left(2*len+2,INF);
32  vector<ll> right(2*len+2,-INF);
33  for(int i=1;i<=2*len+1;++i){
34      left[i+(p[i]-1)]=min(left[i+(p[i]-1)],(ll)i);
35      right[i-(p[i]-1)]=max(right[i-(p[i]-1)],(ll)i);
36  }
37  for(int i=1;i<=2*len+1;++i){
38      right[i]=max(right[i-1],right[i]);
39  }
40  for(int i=2*len;i>=1;--i){
41      left[i]=min(left[i+1],left[i]);
42  }
43  ll ans=0;
44  for(int i=3;i<=2*len-1;i=i+2){
45      ans=max(ans,right[i]-left[i]);
46  }
47  cout<<ans<<"\n";
48  }

```